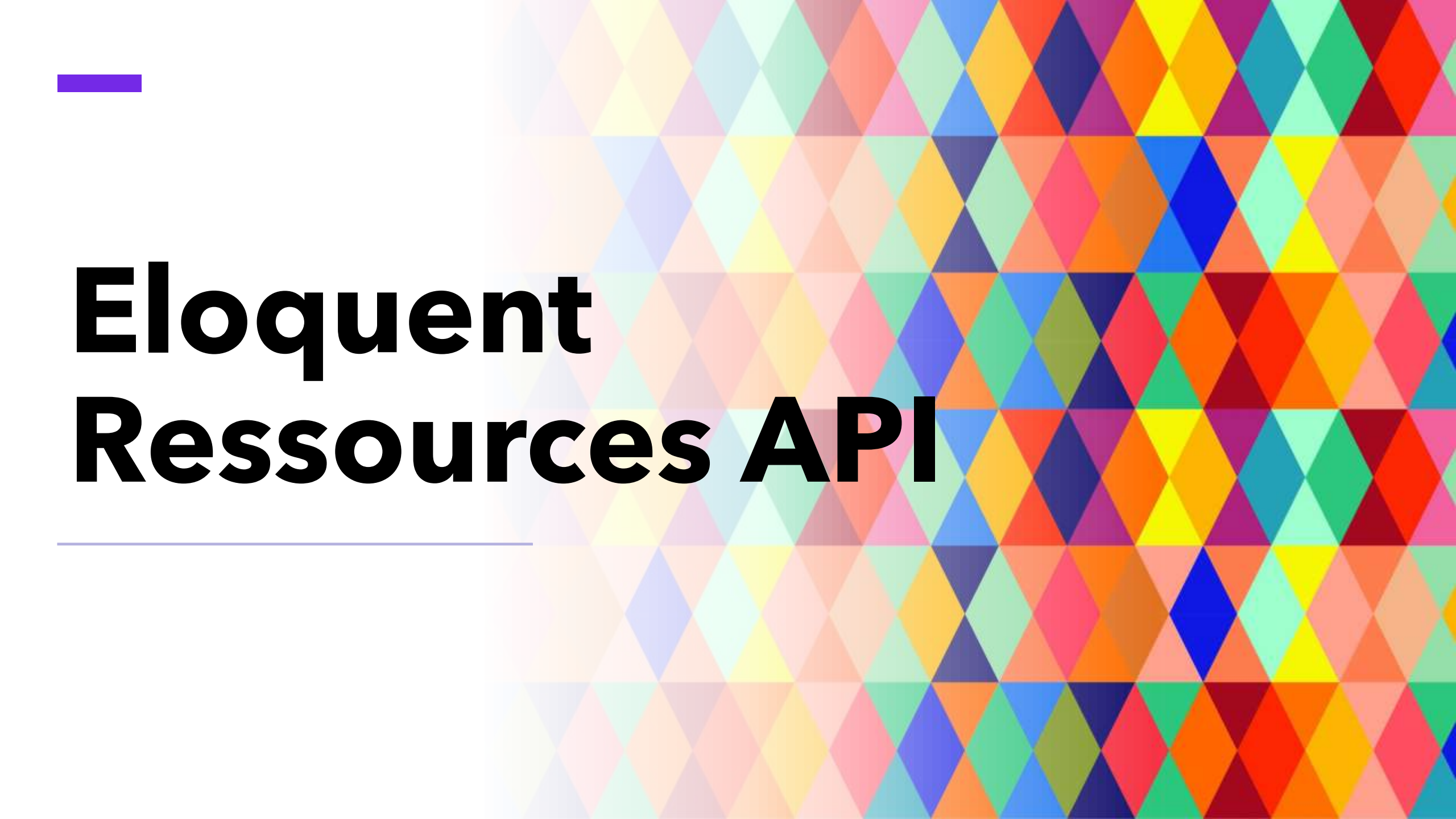
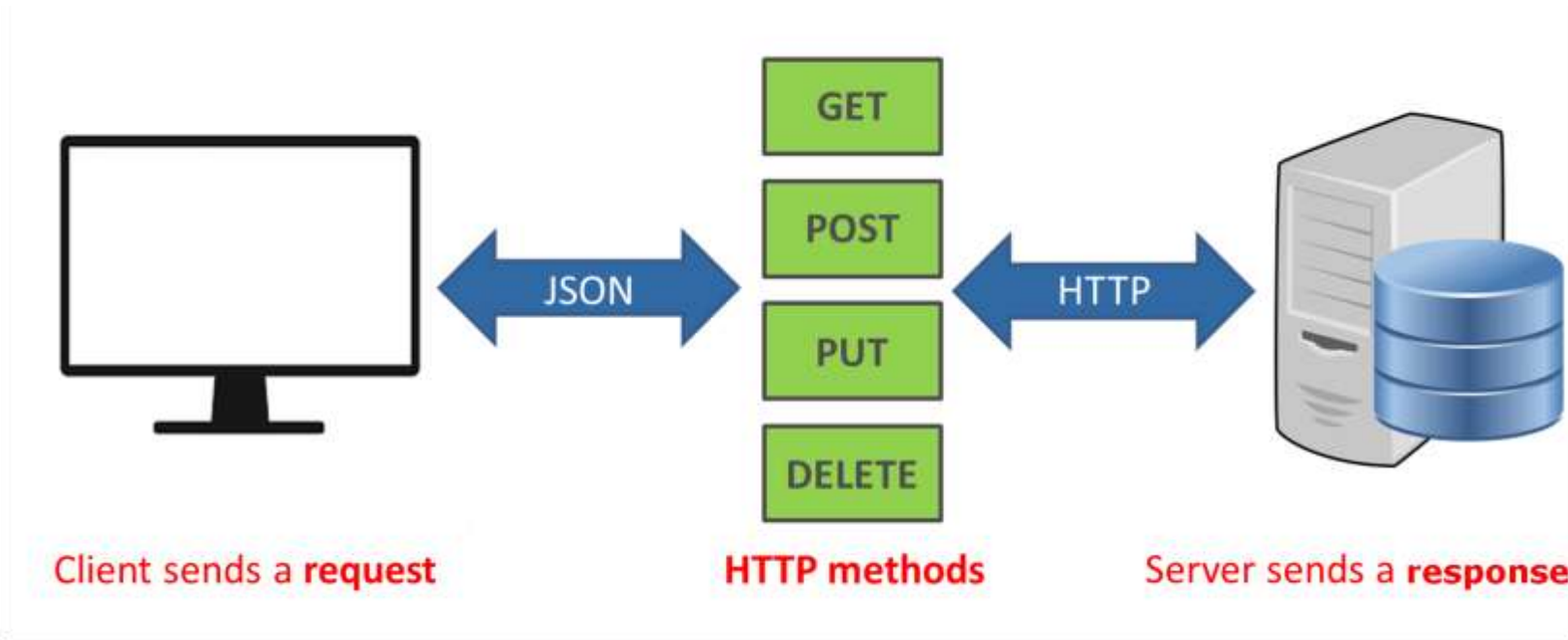




Eloquent Resources API

Introduction

Lorsque vous construisez une API, vous pouvez avoir besoin d'une couche de transformation qui se situe entre vos modèles Eloquent et les réponses JSON qui sont effectivement renvoyées aux utilisateurs de votre application.



Introduction

Vous pouvez toujours convertir les modèles ou collections Eloquent en JSON en utilisant leurs méthodes **toJson** ; cependant, les ressources Eloquent fournissent un contrôle plus granulaire et robuste sur la sérialisation JSON de vos modèles et de leurs relations.



Laravel

Générer des ressources

Pour générer une classe de ressources, vous pouvez utiliser la commande **make:resource**. Par défaut, les ressources seront placées dans le répertoire **app/Http/Resources** de votre application. Les ressources étendent la classe **Illuminate\Http\Resources\Json\JsonResource** :

```
php artisan make:resource UserResource
```

Collections de ressources

En plus de générer des ressources qui transforment des modèles individuels, vous pouvez générer des ressources qui sont responsables de la transformation de collections de modèles. Cela permet à vos réponses JSON d'inclure des liens et d'autres méta-informations pertinentes pour une collection entière d'une ressource donnée.

```
php artisan make:resource User --collection
```

```
php artisan make:resource UserCollection
```

Le concept API

Une classe de ressources représente un modèle unique qui doit être transformé en une structure JSON.

```
class UserResource extends JsonResource
{
    public function toArray($request)
    {
        return [
            'id' => $this->id,
            'name' => $this->name,
            'email' => $this->email,
            'created_at' => $this->created_at,
            'updated_at' => $this->updated_at,
        ];
    }
}
```

Le concept API

Chaque classe de ressource définit une méthode **toArray** qui renvoie le tableau des attributs qui doivent être convertis en **JSON** lorsque la ressource est renvoyée comme réponse à partir d'une méthode de route ou de contrôleur.

Notez que nous pouvons accéder aux propriétés du modèle directement à partir de la variable **\$this**. Cela s'explique par le fait qu'une classe de ressources proxye automatiquement l'accès aux propriétés et aux méthodes vers le modèle sous-jacent afin de faciliter l'accès.

Le concept API

Une fois que la ressource est définie, elle peut être renvoyée par une route ou un contrôleur. La ressource accepte l'instance du modèle sous-jacent via son constructeur :

```
use App\Http\Resources\UserResource;  
use App\Models\User;  
  
Route::get('/user/{id}', function ($id) {  
    return new UserResource(User::findOrFail($id));  
});
```


Collections de ressources

Si vous renvoyez une collection de ressources ou une réponse paginée, vous devez utiliser la méthode de collection fournie par votre classe de ressource lors de la création de l'instance de ressource dans votre route ou votre contrôleur :

```
Route::get('/users', function () {  
    return UserResource::collection(User::all());  
});
```

Collections de ressources

Si vous souhaitez *personnaliser la réponse de la collection* de ressources, vous pouvez créer une ressource dédiée pour représenter la collection :

```
php artisan make:resource UserCollection
```

Collections de ressources

Une fois que la classe collection de ressources a été générée, vous pouvez facilement définir les métadonnées qui doivent être incluses dans la réponse :

```
class UserCollection extends ResourceCollection
{
    public function toArray($request)
    {
        return [
            'data' => $this->collection,
            'links' => [
                'self' => 'link-value',
            ],
        ];
    }
}
```

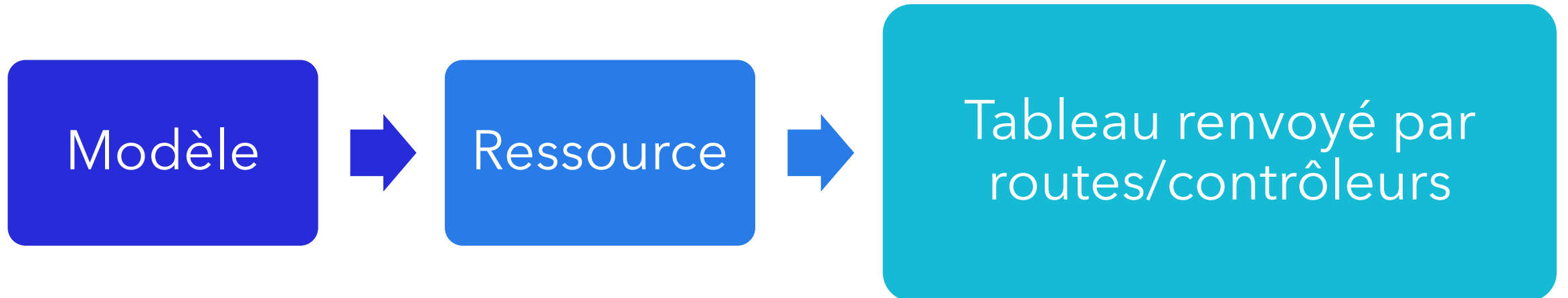
Collections de ressources

Après avoir défini votre collection de ressources, elle peut être renvoyée par une route ou un contrôleur :

```
Route::get('/users', function () {  
    return new UserCollection(User::all());  
});
```

Écriture des ressources

Par essence, les ressources sont simples. Elles doivent seulement transformer un modèle donné en un tableau. Ainsi, chaque ressource contient une méthode **toArray** qui traduit les attributs de votre modèle en un tableau convivial pour l'API qui peut être renvoyé par les routes ou les contrôleurs de votre application



Écriture des ressources

Une fois qu'une ressource a été définie, elle peut être renvoyée directement depuis une route ou un contrôleur :

```
Route::get('/user/{id}', function ($id) {  
    return new UserResource(User::findOrFail($id));  
});
```

Les relations

Si vous souhaitez inclure des ressources liées dans votre réponse, vous pouvez les ajouter au tableau renvoyé par la méthode **toArray** de votre ressource.

```
public function toArray($request)
{
    return [
        'id' => $this->id,
        'name' => $this->name,
        'email' => $this->email,
        'posts' => PostResource::collection($this->posts),
        ...];
}
```

L'enveloppement "data"

Par défaut, votre ressource la plus externe est enveloppée par "data" lorsque la réponse de la ressource est convertie en JSON. Ainsi, par exemple, une réponse typique de ressources ressemble à ce qui suit :

```
{
  "data": [
    {
      "id": 1,
      "name": "Eladio Schroeder Sr.",
      "email": "therese28@example.com"
    },...
  ]
}
```


L'enveloppement "data"

Si vous souhaitez utiliser une clé personnalisée à la place des données, vous pouvez définir un attribut \$wrap sur la classe de ressource :

```
class UserResource extends JsonResource
{

    public static $wrap = 'user';

}
```

L'enveloppement "data"

Si vous souhaitez désactiver l'habillage de la ressource la plus externe, vous devez invoquer la méthode **withoutWrapping** de la classe de base **JsonResource**. En général, vous devez appeler cette méthode à partir de votre **AppServiceProvider**.

```
public function boot()  
{  
    JsonResource::withoutWrapping();  
}
```

Pagination

Vous pouvez passer une instance de **paginator** à la méthode **collection** d'une ressource ou à une collection de ressources personnalisée :

```
Route::get('/users', function () {  
    return new UserCollection(User::paginate());  
});
```

Pagination

Les réponses paginées contiennent toujours des clés méta et des clés de liens avec des informations sur l'état du paginateur :

```
{
  "data": [
    {
      "id": 1,
      "name": "Eladio Schroeder Sr.",
      "email": "therese28@example.com"
    },
    {
      "id": 2,
      "name": "Liliana Mayert",
      "email": "evandervort@example.com"
    }
  ],
}
```

```
"links":{
  "first": "http://example.com/pagination?page=1",
  "last": "http://example.com/pagination?page=1",
  "prev": null,
  "next": null
},
"meta":{
  "current_page": 1,
  "from": 1,
  "last_page": 1,
  "path": "http://example.com/pagination",
  "per_page": 15,
  "to": 10,
  "total": 10
}
```

Attributs conditionnels

Parfois, vous souhaitez inclure un attribut dans la réponse d'une ressource uniquement si une condition donnée est remplie.

```
public function toArray($request)
{
    return [
        'id' => $this->id, ...
        'secret' => $this->when($request->user()->isAdmin(), 'secret-value'),
        'created_at' => $this->created_at,
    ];
}
```